

Assignment 1: Network Latencies, Ping, and Traceroute

Your Name
Your Purdue ID

CS 422 – Fall 2026

Repository

The complete source code, input data, and instructions needed to reproduce the experiments and plots are available at:

<https://github.com/RohMalpani/CS422Assignments>

Reproduction command:

```
./run_all.sh
```

1 Ping Test and Round-Trip Time

1.1 Method

We tested every destination listed in the provided IP-address file, along with our own public IP address. [Describe the rest of the procedure here.]

The code used to collect and process the ping measurements is available here: [ping collection and RTT calculation](#).

1.2 Results

Table 1: Example format for selected ping results.

Destination IP	Distance (km)	Min RTT (ms)	Avg RTT (ms)	Max RTT (ms)
Add results here				

1.3 Distance versus RTT

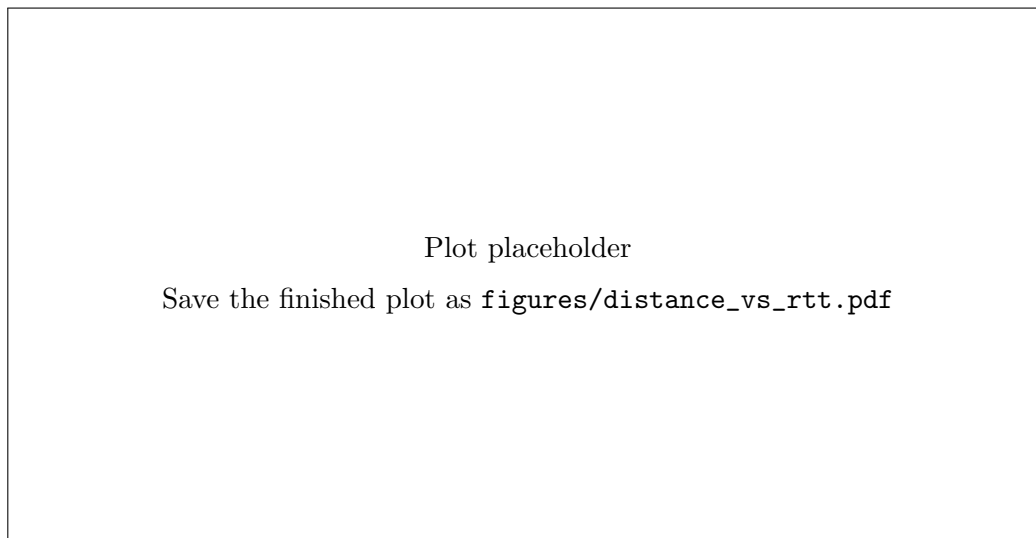


Figure 1: Geographic distance from West Lafayette, Indiana, versus round-trip time. Each point represents one destination IP address.

The plotting code is available here: [distance-versus-RTT plot](#).

1.4 Discussion

[Explain how distance relates to RTT. Discuss propagation delay, routing, congestion, queueing, processing delay, and any important outliers. Explain what the minimum and maximum RTT values convey.]

2 Latency Breakdown

2.1 Method

Five destinations were selected uniformly at random (seed 42, for reproducibility) from the full iperf3 server list.

Table 2: Five destinations selected for traceroute analysis.

Destination IP	Site/Country	Outcome
185.152.67.2	Los Angeles, US	Non-responsive after retry (skipped)
89.187.160.1	Tokyo, JP	Reached, 13 hops
105.235.237.2	Bata, GQ	Reached, 22 hops
103.146.200.98	Port Moresby, PG	Reached, 22 hops
speed2.fiberby.dk	Copenhagen, DK	Reached, 15 hops

Traceroute was run in ICMP-echo mode (-I), sending 3 probes per hop with a 2-second per-probe timeout, up to 30 hops. Per-hop RTT is the *minimum* of the responsive probes at that hop – this approximates the hop’s propagation/processing floor and is less skewed by one transient

queuing delay than an average would be. A hop where all 3 probes time out (shown as * * *) has no RTT value; it is still counted toward hop count but excluded from the per-hop RTT data, since traceroute gives us nothing to measure there. A destination itself is considered non-responsive, and skipped, only if a full traceroute attempt is retried once and still never receives a reply from the destination IP – this distinguishes a genuinely dead/filtered destination from a one-off transient failure (both were observed during testing; see repo `DECISIONS.md` for detail). Initial testing with the default UDP-mode traceroute under-reported reachability: 3 of the first 5 randomly-selected destinations appeared non-responsive, yet a plain `ping` showed all 3 were alive – they (or a router along the path) silently drop the ICMP "port unreachable" reply UDP-mode traceroute depends on, while answering ICMP echo normally. Switching to ICMP-mode traceroute resolved this.

The traceroute collection and parsing code is available here: [traceroute invocation, parsing, and reachability check](#), with the retry/skip logic here: [batch runner with retry-once-before-skip](#). Target selection is here: [seeded random target selection](#).

2.2 Per-Hop Latency Breakdown

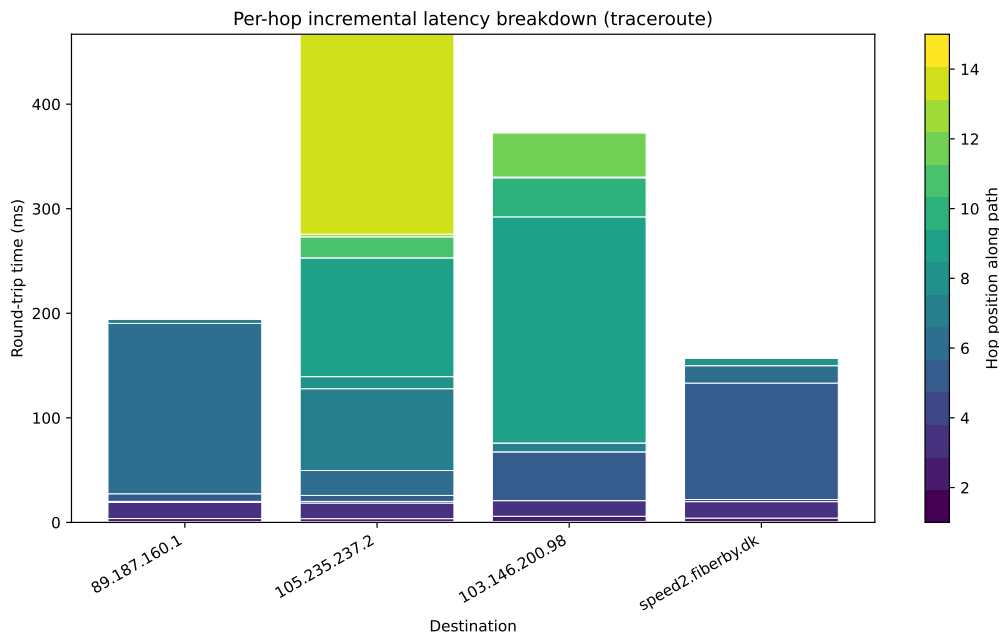


Figure 2: Stacked latency breakdown for the responsive hops along the paths to the five selected destinations.

The plotting code is available here: [stacked per-hop latency chart](#), including how incremental (not cumulative) latency is computed: `incremental_latencies()`.

2.3 Hop Count versus RTT

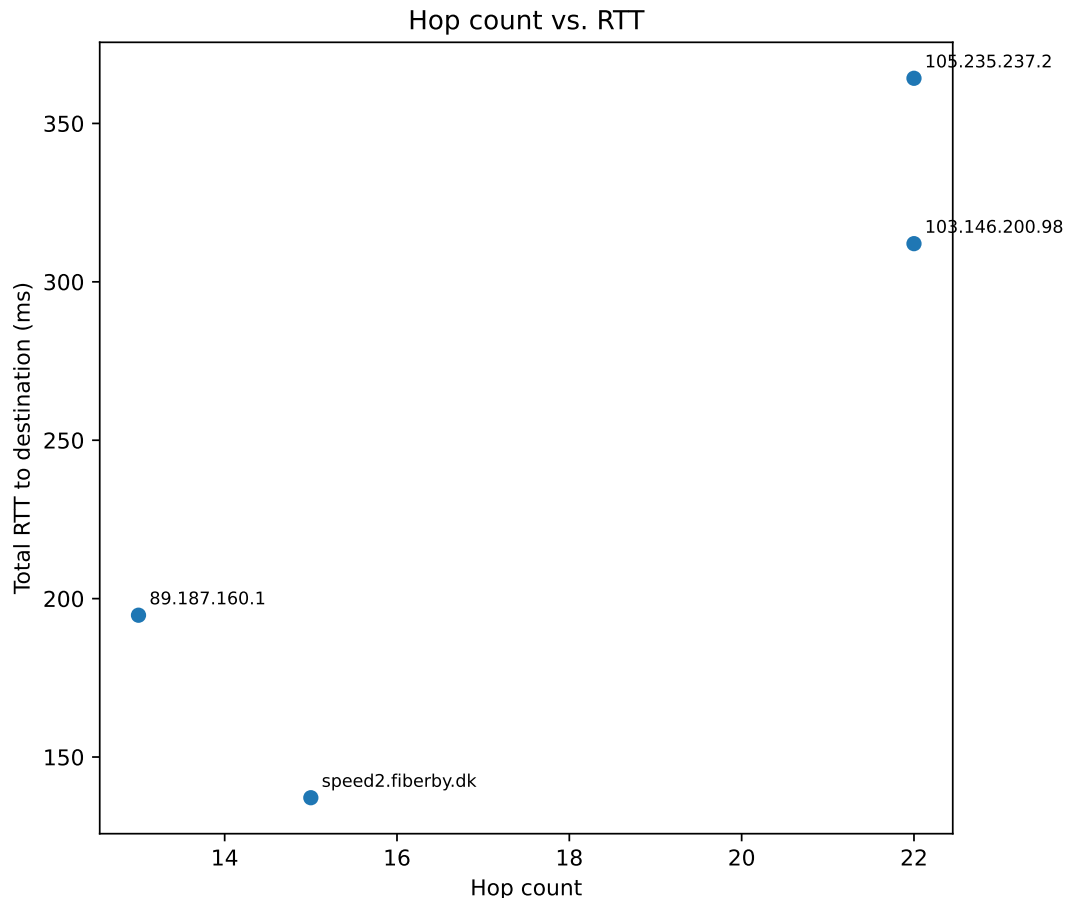


Figure 3: Hop count versus destination RTT. Each point represents one of the five selected destination IP addresses.

The plotting code is available here: [hop-count-versus-RTT plot](#).

2.4 Discussion

Pattern in the stacked bar chart. Across all four reached destinations, the first four hops – our local network and ISP egress in West Lafayette – are uniformly cheap (under 25 ms) regardless of which destination is being traced, since that segment of the path is identical for every trace. Latency then accumulates almost entirely in one or two “long-haul” hops rather than being spread evenly across the path. For example, the trace to Tokyo (89.187.160.1) jumps from 27 ms at hop 9 to 190 ms at hop 10 – a single 163 ms increment, roughly 84% of the entire 194 ms path RTT, almost certainly the trans-Pacific submarine link. The Copenhagen trace shows the same shape at hop 10 (a jump from the last known 21 ms up to 133 ms), consistent with a trans-Atlantic hop. This confirms that a stacked-bar breakdown is genuinely more informative than the raw scatter alone: it shows *where* along the path the latency is coming from, not just the total.

Several hops in every trace returned * * * (fully non-responsive, most often hops 5–8). These are very likely backbone/peering routers that filter ICMP probes but still forward traffic normally

– the traceroute still completes to the destination in every reached case. Because these hops have no RTT of their own, whatever delay they actually contributed is unavoidably folded into the next *responsive* hop’s incremental value (see `incremental_latencies()` above) – a real limitation of traceroute-based measurement, not an error in the analysis, and part of why a single hop’s bar segment can look larger than the physical link itself would suggest.

Hop count versus RTT. The scatter plot shows a rough positive trend – more hops generally means more RTT, since a longer hop count is usually a signature of a physically longer path – but the relationship is weak, not deterministic. Bata, GQ (105.235.237.2) and Port Moresby, PG (103.146.200.98) both take 22 hops, yet their RTTs differ by roughly 50 ms (364 ms vs. 312 ms) – a difference in specific routing (the Bata trace detours through several US-based addresses before reaching Africa) and link quality rather than hop count. Conversely, Tokyo (89.187.160.1) reaches in only 13 hops but has a higher RTT (195 ms) than the 15-hop Copenhagen trace (137 ms), because it is physically farther and crosses a longer undersea link. In short: hop count is a weak proxy for path length, but the *physical distance and quality of the specific long-haul links traversed* – not the number of hops – is what actually determines RTT. This also mirrors what min/max RTT indicated in Part 1: RTT is set mainly by propagation delay over distance, with hop count and instantaneous network state (congestion, queuing at a particular router) contributing second-order variation on top of that floor.

3 Conclusion

[Briefly summarize the main relationship observed among geographic distance, hop count, and RTT.]